

DATA ENGINEERING COURSE

NumPy

Numerical Python

Class Notes

Instructor: Tanishq Tyagi

Data Engineering Course

Table of Contents

- 1. Session Overview**
- 2. Learning Objectives**
- 3. Part 1 — Introduction & Why NumPy?**
 - o What is NumPy?
 - o Limitations of Python Lists
 - o Why NumPy? Core Advantages
- 4. Part 2 — NumPy Arrays**
 - o Creating NumPy Arrays
 - o Array Attributes
 - o Special Array Creation Functions
- 5. Part 3 — Operations & Broadcasting**
 - o Element-wise Arithmetic
 - o Aggregate Functions
 - o Broadcasting
- 6. Part 4 — Hands-on Practical**
- 7. Part 5 — Performance Comparison**
- 8. Real-World Data Engineering Perspective**
- 9. Summary & Quick Revision Sheet**
- 10. Interview Questions — Beginner & Intermediate**
- 11. Data Engineer Interview Preparation**

1. Session Overview

This session introduced **NumPy (Numerical Python)**, one of the most fundamental libraries in the Python data ecosystem. The class began with a quick recap, then explored why plain Python lists fall short for large-scale numerical work, followed by a deep dive into NumPy arrays, operations, broadcasting, and a live performance benchmark. By the end of the session, students had built and analysed arrays, applied vectorised operations, and measured the speed difference between a Python loop and a NumPy operation on one million numbers.

Part	Topic	Key Focus
Part 0	Recap & Quiz	Previous session concepts
Part 1	Introduction & Why NumPy?	Limitations of lists, NumPy advantages
Part 2	NumPy Arrays	Creating arrays, attributes, special functions
Part 3	Operations & Broadcasting	Arithmetic, aggregates, broadcasting
Part 4	Hands-on Practical	Student Performance Analyzer
Part 5	Performance Comparison	List vs NumPy speed benchmark

2. Learning Objectives

By the end of this session, students will be able to:

- Explain what NumPy is and why it is preferred over plain Python lists for numerical work.
- Understand how NumPy stores data in contiguous memory and why that makes it faster.
- Create 1-D and 2-D NumPy arrays from Python lists and using built-in functions (`np.zeros`, `np.ones`, `np.arange`).
- Inspect array attributes: `shape`, `size`, `ndim`, and `dtype`.
- Perform element-wise arithmetic (`+`, `-`, `*`, `/`) on NumPy arrays.
- Use aggregate functions: `max()`, `min()`, `mean()`, `sum()`.
- Apply broadcasting to perform scalar operations on arrays without loops.
- Benchmark Python list operations against NumPy operations and interpret the results.
- Connect NumPy concepts to real-world Data Engineering pipelines and ETL workflows.
- Answer beginner-to-intermediate interview questions on NumPy.

3. Part 1 — Introduction & Why NumPy?

3.1 What is NumPy?

NumPy stands for **Numerical Python**. It is an open-source Python library that provides support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on them. NumPy is the foundational package for scientific computing in Python and forms the backbone of libraries such as Pandas, Scikit-learn, TensorFlow, and SciPy.

Key use cases of NumPy:

- Fast numerical computations on large datasets.
- Scientific computing and statistical analysis.
- Matrix and linear algebra operations.
- Machine Learning model computations (feature matrices, weight updates).
- Data analysis and preprocessing pipelines.

3.2 Limitations of Python Lists

To understand *why* NumPy exists, we first need to appreciate what Python lists cannot do well.

3.2.1 Python Lists are Heterogeneous

A Python list can hold any type of object at once. While this is flexible, it comes at a cost:

```
my_list = ['Tanishq', 1, 2, 3, 5, True, 5.0]
# Strings, integers, booleans, and floats all in one list
```

3.2.2 Object Overhead

Python lists do not store values directly. Instead, they store *references* (pointers) to Python objects. Each object carries extra metadata (type, reference count, etc.). For a list of 5 million salaries, Python creates 5 million separate objects in memory — a huge overhead that slows down computation.

3.2.3 Sequential (Loop-based) Processing

Python lists process elements one by one using interpreted Python loops. There is no built-in way to operate on all elements at once. For example, multiplying every salary in a 5-million-row list by 1.1 requires a for loop — slow for large datasets.

*** Real-world scenario: Imagine a CSV file containing salaries of 5 million employees. With a plain Python list, applying a 10% raise requires a loop — this is both slow and memory-inefficient.**

3.3 Why NumPy? Core Advantages

Advantage	Explanation
Homogeneous data	All elements share the same data type → no type-checking overhead per element.
Contiguous memory	NumPy stores all elements in a single block of memory → cache-friendly access.
No object overhead	Values stored directly, not as Python objects with metadata.
C-optimised code	Heavy computations run in compiled C code, not interpreted Python.
Vectorisation	Operations applied to entire arrays at once without Python loops.
Broadcasting	Scalar/array operations without explicit loops.

4. Part 2 — NumPy Arrays

4.1 Importing NumPy

NumPy is imported using the conventional alias **np**. This alias is a community standard; all NumPy code you will encounter uses it.

```
import numpy as np
```

4.2 Creating NumPy Arrays

4.2.1 From a Python List — `np.array()`

The most common way to create a NumPy array is by passing a Python list to **`np.array()`**.

```
import numpy as np

# 1-D array
arr = np.array([10, 20, 30, 40, 50])
print(type(arr)) # <class 'numpy.ndarray'>

# 2-D array (matrix)
arr_2d = np.array([[10, 20, 30],
                   [40, 50, 60]])
```

The object returned is an **`ndarray`** — short for *N-Dimensional Array*. This is the core data structure of NumPy.

4.3 Array Attributes

Every NumPy array exposes a set of attributes that describe its structure and data type. These are essential for debugging and for writing dimension-aware code.

```
arr_2d = np.array([[10, 20, 30],
                   [40, 50, 60]])

print(arr_2d.shape) # (2, 3) → 2 rows, 3 columns
print(arr_2d.ndim) # 2 → 2-dimensional
print(arr_2d.size) # 6 → total number of elements
print(arr_2d.dtype) # int64 → 64-bit integer
```

Attribute	Returns	Meaning
<code>.shape</code>	tuple	Dimensions of the array (rows x columns x ...)
<code>.ndim</code>	int	Number of dimensions

.size	int	Total number of elements
.dtype	dtype	Data type of the elements (int64, float64, etc.)

4.4 Special Array Creation Functions

4.4.1 np.arange() — Range-based Arrays

np.arange() works like Python's built-in `range()` but returns a NumPy array.

```
# All integers from 0 to 9
arr = np.arange(10)
print(arr) # [0 1 2 3 4 5 6 7 8 9]

# Even numbers from 2 to 8 (start, stop, step)
arr = np.arange(2, 10, 2)
print(arr) # [2 4 6 8]
```

4.4.2 np.zeros() — Array of Zeros

```
zero_arr = np.zeros(5, dtype=int)
print(zero_arr) # [0 0 0 0 0]
print(zero_arr.dtype) # int64
```

Use case: initialising accumulators, padding arrays, or representing missing/default values.

4.4.3 np.ones() — Array of Ones

```
one_arr = np.ones(5, dtype=int)
print(one_arr) # [1 1 1 1 1]
print(one_arr.dtype) # int64
```

Use case: creating mask arrays, initialising weight matrices in ML, status flags.

*** Quick tip: You can pass `dtype=float` to any of these functions to change the element type. Default dtype is `float64` for `zeros/ones` and inferred for `np.array()`.**

5. Part 3 — Operations & Broadcasting

5.1 Element-wise Arithmetic

NumPy supports **element-wise arithmetic** operations directly on arrays. Each operation is applied to every corresponding pair of elements without writing a loop.

```
import numpy as np

list_a = [1, 2, 3]
list_b = [1, 2, 3]

# Python list addition → concatenation (NOT numeric addition)
print(list_a + list_b) # [1, 2, 3, 1, 2, 3]

numpy_a = np.array(list_a)
numpy_b = np.array(list_b)

print(numpy_a + numpy_b) # [2 4 6]
print(numpy_a - numpy_b) # [0 0 0]
print(numpy_a * numpy_b) # [1 4 9]
print(numpy_a / numpy_b) # [1. 1. 1.]
```

! Important: Adding two Python lists with '+' concatenates them. NumPy '+' adds element by element. This is one of the most common beginner mistakes.

5.2 Aggregate Functions

Aggregate functions collapse an entire array (or axis) into a single scalar value.

```
arr = np.array([1, 2, 3])

print(arr.max()) # 3
print(arr.min()) # 1
print(arr.mean()) # 2.0
print(arr.sum()) # 6

# Formatted output
print(f'max: {arr.max()}')
print(f'min: {arr.min()}')
print(f'mean: {arr.mean()}')
print(f'sum: {arr.sum()}')
```

Method	Description	Example (arr=[1,2,3])
--------	-------------	-----------------------

.max()	Maximum value	3
.min()	Minimum value	1
.mean()	Arithmetic mean	2.0
.sum()	Sum of all elements	6
.std()	Standard deviation	0.816...
.var()	Variance	0.666...

5.3 Vectorisation & Broadcasting

5.3.1 What is Broadcasting?

Broadcasting is NumPy's mechanism for applying a scalar (or smaller array) operation to every element of a larger array — without writing a Python loop and without creating a temporary copy of the scalar. NumPy conceptually 'stretches' the scalar across the array shape.

```
marks = np.array([90, 85, 80, 75])

# Add 5 grace marks to every student – no loop needed
print(marks + 5) # [95 90 85 80]

# Increase every mark by 10%
print(marks * 1.10) # [99. 93.5 88. 82.5]
```

*** Broadcasting rule: NumPy compares shapes element-by-element from the right. Dimensions are compatible when they are equal or one of them is 1.**

6. Part 4 — Hands-on Practical

The following example walks through the **Student Performance Analyzer** — the class assignment. Each step is explained in detail.

```
import numpy as np
import random

# Step 1: Create an array of 8 random marks (0-100)
marks = np.array([random.randint(40, 100) for _ in range(8)])
print('Marks:', marks)

# Step 2: Array attributes
print('Shape :', marks.shape) # (8,)
print('Size :', marks.size) # 8
print('Ndim :', marks.ndim) # 1
print('Dtype :', marks.dtype) # int64

# Step 3: Aggregate statistics
print('Total :', marks.sum())
print('Average :', marks.mean())
print('Highest :', marks.max())
print('Lowest :', marks.min())

# Step 4: Add 5 grace marks to every student (broadcasting)
marks_grace = marks + 5
print('After grace:', marks_grace)

# Step 5: Increase every mark by 10%
marks_raised = marks * 1.10
print('After 10% raise:', marks_raised)

# Step 6: Special arrays
zeros_arr = np.zeros(8, dtype=int) # [0 0 0 0 0 0 0 0]
ones_arr = np.ones(8, dtype=int) # [1 1 1 1 1 1 1 1]
roll_nos = np.arange(1, 9) # [1 2 3 4 5 6 7 8]

print('Zeros:', zeros_arr)
print('Ones :', ones_arr)
print('Rolls:', roll_nos)
```

Line-by-line explanation:

Line 5 — List comprehension + `np.array()`: Generates 8 random integers and wraps them in a NumPy array.

Line 9 — `.shape`: Returns a tuple. For a 1-D array of 8 elements: `(8,)`.

Line 10 — `.size`: Total number of elements = 8.

Line 11 — `.ndim`: Number of axes/dimensions = 1 (flat array).

Line 12 — `.dtype`: NumPy infers `int64` for integer inputs on most systems.

Lines 15-18 — Aggregates: `sum()`, `mean()`, `max()`, `min()` are called as methods on the array.

Line 21 — `marks + 5`: Broadcasting: 5 is added to every element in one C-level operation.

Line 25 — `marks * 1.10`: Scalar multiplication; output dtype promotes to `float64`.

Lines 28-30 — Special arrays: `zeros`, `ones`, and `arange` are the three most-used factory functions.

7. Part 5 — Performance Comparison & Wrap-up

One of NumPy's biggest selling points is raw speed. The following benchmark multiplies one million numbers by 2 using (a) a Python list with a for loop and (b) a NumPy array.

```
import numpy as np
import time

# Setup
numbers = list(range(1_000_000)) # Python list
array = np.arange(1_000_000) # NumPy array

# --- Python List with for loop ---
result = []
start = time.time()
for x in numbers:
    result.append(x * 2)
list_time = time.time() - start

# --- NumPy vectorised operation ---
start = time.time()
result = array * 2
numpy_time = time.time() - start

print(f'List time : {list_time:.5f} seconds')
print(f'NumPy time: {numpy_time:.5f} seconds')
```

Typical output:

```
List time : 0.08500 seconds
NumPy time: 0.00120 seconds
```

Why is NumPy faster?

- NumPy operations run in **pre-compiled C code**, bypassing the Python interpreter overhead.
- Data is stored in a **contiguous memory block** — the CPU cache can prefetch values efficiently.
- There is **no Python object creation** per element — no reference counting, no type boxing.
- Modern CPUs can apply **SIMD (Single Instruction, Multiple Data)** vectorisation — one instruction processes multiple data points simultaneously.

*** Rule of thumb: NumPy is typically 10x-100x faster than an equivalent Python loop for numerical operations on arrays of 100,000+ elements.**

8. Real-World Data Engineering Perspective

8.1 Why Data Engineers Use NumPy

Data Engineers build and maintain systems that ingest, transform, and serve data at scale. NumPy appears throughout the data engineering stack:

ETL / ELT Pipelines

When transforming raw data — normalising salary columns, applying inflation adjustments, converting currencies — NumPy's vectorised operations replace slow Python loops.

Pandas Integration

Pandas DataFrames are built on top of NumPy arrays. Understanding NumPy helps you write efficient Pandas code and debug dtype-related issues.

Feature Engineering for ML Pipelines

Data Engineers prepare feature matrices (2-D arrays) for ML models. NumPy is the standard container for these matrices.

Statistical Quality Checks

Aggregate functions (mean, std, min, max) are used in data-quality pipelines to detect anomalies, outliers, or data drift in production.

Performance-critical Microservices

In real-time pipelines (e.g., fraud detection, recommendation scoring), NumPy operations run inside Python microservices handling thousands of requests per second.

8.2 Where NumPy Appears in Data Pipelines

Pipeline Stage	NumPy Role
Data Ingestion	Reading CSVs into arrays, dtype coercion
Data Cleaning	Replacing nulls with zeros/means, clipping outliers
Transformation	Scaling, normalisation, encoding — all vectorised
Feature Engineering	Creating derived columns, polynomial features
Model Serving	Input/output arrays for ML inference
Monitoring	Computing aggregate metrics for dashboards

9. Summary & Quick Revision Sheet

9.1 Key Concepts Summary

NumPy ndarray: The core N-dimensional array object. Homogeneous, contiguous memory.

np.array(): Creates an array from a Python list or nested lists.

Attributes: .shape (dimensions), .ndim (num axes), .size (total elements), .dtype (element type).

np.zeros(n): Array of n zeros. Great for initialisation.

np.ones(n): Array of n ones. Used for masks and defaults.

np.arange(start, stop, step): Range-based array. Returns values in [start, stop).

Element-wise ops: + - * / between same-shape arrays operate element by element.

Aggregates: .max() .min() .mean() .sum() — collapse array to a scalar.

Broadcasting: Scalar operations applied to every element without a loop.

Vectorisation: Operations dispatched to C code — much faster than Python loops.

9.2 Syntax Cheat Sheet

```
import numpy as np

arr = np.array([1, 2, 3]) # Create from list
arr = np.zeros(5, dtype=int) # [0 0 0 0 0]
arr = np.ones(5, dtype=int) # [1 1 1 1 1]
arr = np.arange(1, 6) # [1 2 3 4 5]
arr = np.arange(0, 10, 2) # [0 2 4 6 8]

arr.shape # (n,) or (rows, cols)
arr.ndim # 1 or 2 ...
arr.size # total elements
arr.dtype # int64, float64 ...

arr.max() arr.min() arr.mean() arr.sum()

arr + 5 # broadcasting: add 5 to every element
arr * 1.1 # broadcasting: multiply every element by 1.1
a + b # element-wise addition (same shape required)
```

9.3 Common Mistakes

- Using + on two lists expecting numeric addition — Python concatenates lists.
- Forgetting that np.arange(stop) starts at 0, not 1.

- Mixing dtypes — NumPy upcasts silently (int + float → float).
- Assuming `.shape` returns an integer — it returns a tuple.
- Confusing `.size` (total elements) with `len()` (first dimension only).

10. Interview Questions

10.1 Beginner Level

Q1. What does NumPy stand for?

NumPy stands for Numerical Python. It is an open-source library for fast numerical computing in Python.

Q2. What is an ndarray?

ndarray stands for N-Dimensional Array. It is the core data structure of NumPy — a homogeneous, fixed-type, contiguous array that can have any number of dimensions.

Q3. How do you import NumPy?

`import numpy as np` — the alias 'np' is a universal community convention.

Q4. How is a NumPy array different from a Python list?

NumPy arrays are homogeneous (one data type), stored in contiguous memory, and support vectorised operations. Python lists are heterogeneous, store object references, and require loops for element-wise operations.

Q5. What does `.shape` return?

A tuple representing the size of each dimension. For a 2-row, 3-column array: (2, 3). For a flat array of 5 elements: (5,).

Q6. What is `.ndim`?

The number of dimensions (axes) of the array. A flat array is 1-D; a matrix is 2-D; a tensor is 3-D.

Q7. What does `.size` return?

The total number of elements across all dimensions. For a (2,3) array: size = 6.

Q8. What is `.dtype`?

The data type of the elements, e.g., `int64`, `float64`, `bool`. All elements share the same dtype.

Q9. How do you create an array of zeros?

`np.zeros(n)` returns an array of n float zeros. Pass `dtype=int` for integer zeros.

Q10. What does `np.arange(1, 9)` return?

An array `[1 2 3 4 5 6 7 8]` — integers from 1 up to (but not including) 9.

Q11. What is broadcasting in NumPy?

Broadcasting is NumPy's ability to apply a scalar or smaller array operation to a larger array element-wise, without an explicit loop. E.g., `arr + 5` adds 5 to every element.

Q12. What does `arr.mean()` return?

The arithmetic mean of all elements in the array.

10.2 Intermediate Level

Q1. Why is NumPy faster than a Python for loop?

NumPy operations are executed in pre-compiled C code, operate on contiguous memory blocks, avoid per-element Python object overhead, and can leverage CPU SIMD instructions. A Python for loop is interpreted one instruction at a time.

Q2. What is vectorisation?

Vectorisation means applying an operation to an entire array at once rather than iterating. In NumPy, `arr * 2` is vectorised — one C-level instruction handles all elements.

Q3. What happens when you add two Python lists with '+'?

Python concatenates them: `[1,2] + [3,4] = [1,2,3,4]`. This is different from NumPy's element-wise addition.

Q4. What is dtype promotion?

When you mix int and float arrays, NumPy automatically promotes the result to float64 to avoid data loss.

Q5. How do you create a 2-D array?

Pass a nested list: `np.array([[1,2,3],[4,5,6]])` creates a (2,3) array.

Q6. What is the difference between `np.zeros(5)` and `np.zeros(5, dtype=int)`?

`np.zeros(5)` returns float64 zeros (`[0. 0. 0. 0. 0.]`). `dtype=int` returns int64 zeros (`[0 0 0 0 0]`).

Q7. How do you increase every element by 10%?

`arr * 1.10` — broadcasting applies the scalar multiplication to every element.

Q8. What is the output of `np.arange(2, 10, 2)`?

`[2 4 6 8]` — starts at 2, steps by 2, stops before 10.

Q9. How does NumPy handle memory differently from Python lists?

NumPy allocates a single contiguous block of memory for all elements. Python lists store an array of pointers to individual Python objects scattered in memory. Contiguous storage improves cache performance dramatically.

Q10. What is the difference between `.size` and `len()` for a 2-D array?

`.size` returns total elements (rows * cols). `len()` returns only the size of the first dimension (number of rows).

Q11. When would you use `np.ones()`?

Common uses: initialising weight arrays (ML), creating Boolean masks (all True), placeholder arrays for accumulation, status flags (e.g., item in stock).

Q12. Can NumPy arrays hold strings?

Yes, using `dtype='U'` (Unicode string), but this loses numerical performance benefits. For mixed data, Pandas DataFrames are the right tool.

11. Data Engineer Interview Preparation

This section focuses on how NumPy concepts are applied in real production data engineering roles. Interviewers at data-intensive companies (fintechs, e-commerce, cloud platforms) regularly ask about performance, scalability, and integration with the broader data stack.

DE-Q1. How does NumPy fit into a typical ETL pipeline?

In an ETL pipeline, NumPy is used during the Transform phase. After ingesting raw data (e.g., a CSV of transaction amounts), a Data Engineer loads it into a NumPy array and applies vectorised transformations — currency conversion, tax calculations, outlier clipping — before loading to a data warehouse. NumPy's speed is critical when processing millions of rows.

DE-Q2. Why would you choose NumPy over pandas for certain tasks?

NumPy has lower overhead than pandas for pure numerical operations on homogeneous data. When you have a single numeric column of 50 million rows and only need to apply arithmetic, a NumPy array is faster and uses less memory than a pandas Series. Pandas is preferred when you need labelled data, mixed types, or SQL-like operations.

DE-Q3. How do you handle missing values in NumPy?

NumPy uses `np.nan` (Not a Number) for missing float values. Functions like `np.nanmean()`, `np.nansum()`, `np.nanmax()` ignore NaN values automatically. For integer arrays, a sentinel value (e.g., -1 or 0) is commonly used since integer arrays cannot hold NaN. In production, Pandas nullable integer types or masking arrays are also options.

DE-Q4. You have a NumPy array of 10 million rows and need to multiply every element by 1.08. How do you do it efficiently?

Simply use `arr * 1.08`. NumPy broadcasts the scalar and executes the multiplication in optimised C code across all 10 million elements in a single vectorised call. Never write a Python for loop for this — it would be 50-100x slower.

DE-Q5. What is dtype and why does it matter in a production pipeline?

dtype determines how each element is stored in memory. `int32` uses 4 bytes/element; `float64` uses 8 bytes/element. For 10 million rows, switching from `float64` to `float32` halves memory usage (80 MB → 40 MB) and can double throughput on SIMD-capable CPUs. Incorrect dtypes can also cause silent numeric overflow — a serious bug in financial systems.

DE-Q6. How would you benchmark a NumPy operation in production?

Use Python's `time` module for quick benchmarks (as shown in class). For rigorous benchmarking, use the `timeit` module or IPython's `%timeit` magic. In production pipelines, instrument with Prometheus metrics or DataDog APM to track p50/p95/p99 latency of critical transformation steps over time.

DE-Q7. What is the role of NumPy in a Machine Learning pipeline managed by a Data Engineer?

Data Engineers prepare the feature matrix (a 2-D NumPy array of shape `[n_samples, n_features]`) that is fed to ML models. They also apply preprocessing (normalisation, one-hot encoding) using NumPy operations. The model's output (predictions) is also a NumPy array that engineers then write back to a database or stream to downstream systems.

DE-Q8. Explain how NumPy's contiguous memory model improves cache performance.

Modern CPUs have multiple cache levels (L1, L2, L3). Cache lines are fetched in blocks of 64 bytes. When NumPy iterates over a contiguous array, each cache fetch loads several consecutive elements, making subsequent accesses instant (cache hits). A Python list of object pointers causes cache misses because the actual values are scattered throughout the heap.

DE-Q9. How would you use NumPy to detect outliers in a sales column?

Compute mean and standard deviation: `mean = arr.mean(); std = arr.std()`. Flag values beyond 3 standard deviations: `outliers = arr[np.abs(arr - mean) > 3 * std]`. This is a vectorised boolean mask operation — no Python loop required.

DE-Q10. What are the scalability limits of NumPy?

NumPy operates on in-memory data on a single machine. For datasets larger than RAM, alternatives include: Dask (parallelises NumPy-like operations across cores/nodes), CuPy (NumPy on GPU), Apache Spark (distributed), or memory-mapped arrays with `np.memmap`. Knowing these limits is expected in senior Data Engineering interviews.

DE-Q11. A colleague writes a loop to add a tax column to 5 million rows. How do you fix it?

Replace the loop with broadcasting: `tax_array = price_array * 0.18`. This runs in compiled C code and completes in milliseconds vs. several seconds for a loop. In a code review, always flag Python loops over large NumPy arrays as a performance anti-pattern.

DE-Q12. How does NumPy relate to Apache Arrow and modern data formats?

Apache Arrow uses a columnar, contiguous memory layout very similar to NumPy. Libraries like PyArrow can share memory with NumPy (zero-copy) via the `__array__` protocol. Parquet files are read into Arrow tables, which can be converted to NumPy arrays with `.to_pydict()` or via Pandas. Understanding NumPy memory layout helps you reason about Arrow performance too.

DE-Q13. What would happen if you apply `arr + 5` on a float64 array vs. an int64 array?

Both operations work. For int64 arr, the result is int64. If 5 is a Python float (5.0), NumPy promotes to float64 to avoid precision loss. Always check dtypes after operations in production to avoid unexpected type promotions that increase memory usage.

DE-Q14. Describe a scenario where NumPy's speed made a critical difference.

Example: A fintech company runs end-of-day settlement for 3 million trades. Each trade amount must be multiplied by a FX rate and rounded to 2 decimal places. Using a Python loop: ~25 seconds. Using NumPy broadcasting + `np.round()`: ~0.3 seconds. The 80x speedup allows the settlement job to complete well within the nightly batch window, avoiding SLA breaches.

DE-Q15. How do you ensure reproducibility when generating random data with NumPy?

Set a random seed: `np.random.seed(42)`. This makes random number generation deterministic. In production data pipelines, reproducibility is critical for debugging, A/B testing, and audit trails. For newer code, prefer `np.random.default_rng(seed=42)` which uses a more modern random generator (PCG64).
